TypeScript

Why TS..?

Main Problem TS Solves: Type Safety & Early Error Detection

Problem in JS:

```
function add(a, b) {
  return a + b;
}

add(5, "10"); // JS allows this → "510" 😱
```

- JS doesn't check types until **runtime**.
- Bugs appear **late**, often in production.
- Refactoring is risky; you never know if you broke something until runtime.

TS Solution:

```
function add(a: number, b: number): number {
  return a + b;
}

add(5, "10"); // ❌ Type error at compile time
```

✅ **Catch errors early, before running the code.**

2️⃣ **Problem: Large-Scale Maintainability**

Problem in JS:

- In big projects, functions, objects, and modules grow massive.
- Hard to know **what type of data** each function expects.
- Team members constantly break each other's code.

TS Solution:

- Types, interfaces, enums, and strict compiler rules make contracts **explicit**.
- Every developer knows what input/output is expected.
- Safe refactoring is possible: rename functions, restructure modules, update features without fear.

3.Problem: Runtime Surprises with Null/Undefined

Problem in JS:

let user = getUser();
console.log(user.name); // ❌ Throws runtime error if user is null

TS Solution:

let user: User | null = getUser();
console.log(user?.name); // ✅ Safe access

- Prevents crashes and undefined behavior.

TypeScript solves the main problems of JavaScript: it catches type errors at compile-time, enforces code contracts, improves maintainability for large projects, enhances tooling and developer productivity, and prevents runtime surprises.

2️⃣ How TypeScript works in the engine

important: Browsers or Node.js do NOT understand TypeScript directly. They only understand JavaScript. So TS needs to compile (transpile) into JS. Here's the flow:

Write TS Code

```
let username: string = "Prajwal";
```

Compile with tsc (TypeScript Compiler)

- Checks **types** at compile-time.

- Converts TS code into **plain JavaScript** that browsers or Node.js can run.

tsc index.ts

1. Output (index.js):

```
var username = "Prajwal";
```

2. **Execution in JS engine**
    - Once transpiled, the JS runs in **V8 engine** (Node.js or Chrome) like any normal JS.
    - TS does **not exist at runtime**; it's purely a dev-time tool for type checking.

So, **TS = development-time safety**, **JS = runtime execution**.

Create TS Project

Step 1: Create Project Folder

```
mkdir my-ts-project
cd my-ts-project
```

Step 2: Initialize npm

```
npm init -y
```

This creates package.json.

Step 3: Initialize TypeScript Project

```
npx tsc --init
```

- **Creates tsconfig.json, which configures the TS compiler.**
- **A basic tsconfig.json looks like this:**

Step 4: Create Source Folder and TS File

```
mkdir src
touch src/index.ts
```

Example index.ts:

```
function greet(name: string): string {
   return `Hello, ${name}!`;
}

console.log(greet("Prajwal"));
```

Step 5: Compile TypeScript

```
npx tsc
```

- TS compiler reads tsconfig.json, compiles src/index.ts → dist/index.js
- Run the JS:

```
node dist/index.js
```

Output:

Hello, Prajwal!

Step 6: Run TS Directly (Optional)

- With **ts-node**, you can run without compiling:

```
npx ts-node src/index.ts
```

Step 7: Add NPM Scripts (Optional)

In package.json:

```
"scripts": {
  "start": "node dist/index.js",
  "build": "tsc",
```

```
  "dev": "ts-node src/index.ts"
}
```

- npm run build → compile
- npm run start → run compiled JS
- npm run dev → run TS directly

3 tsconfig.json main parts

tsconfig.json tells the TS compiler **how to transpile your code**. Some key fields:

```
{
  "compilerOptions": {
    "target": "ES2020",          // JS version to compile to
    "module": "CommonJS",        // Module system: CommonJS (Node), ES6, etc.
    "outDir": "./dist",          // Output folder
    "rootDir": "./src",          // Root TS files folder
    "strict": true,              // Enables all strict type checks
    "esModuleInterop": true,     // Allows default imports from commonjs
    "forceConsistentCasingInFileNames": true, // Case-sensitive imports
    "skipLibCheck": true         // Skip checking node_modules types
  },
  "include": ["src"],            // Files/folders to include
  "exclude": ["node_modules"]    // Files/folders to ignore
}
```

Most important options explained:

- **target** – Which JS version TS outputs (ES5, ES6/ES2015, ES2020…)
- **module** – Module system (CommonJS for Node, ESNext for modern JS)
- **strict** – Turns on all strict checks: noImplicitAny, strictNullChecks, etc.
- **outDir & rootDir** – Controls folder structure in transpiled JS
- **esModuleInterop** – Fixes import/export compatibility between TS & JS modules

How to run nodemon

1️⃣ Best Way (nodemon + ts-node)

Step 1 — Install dependencies

npm install ts-node

- **ts-node → run TS directly**

Add script in package.json

```
"scripts": {
  "dev": "nodemon --exec ts-node src/index.ts"
}
```

Run:

npm run dev

Now whenever you **save a file**, nodemon restarts automatically 🔁

File structuring

A simple TS project usually looks like this:

```
my-ts-project
│
├── node_modules
├── dist
├── src
│   ├── index.ts
│   └── utils.ts
│
├── package.json
├── tsconfig.json
└── nodemon.json
```

Explanation

src/

- **All TypeScript source code This is where developers write code**

In a TypeScript backend project, we usually organize code inside a src folder and compile it into a dist folder. Inside src, we separate concerns using folders like controllers, services, routes, models, middlewares, utils, and types. This structure improves scalability, maintainability, and makes large applications easier to manage.

Primitive Types

1️⃣ What are Primitive Types in TypeScript?

Primitive types are the **basic data types** used to store simple values.

TypeScript mainly has these primitive types:

- string
- number
- boolean
- null
- undefined
- symbol
- bigint

Think of them as the **smallest building blocks of data**.

2️⃣ string

Used to store text.

```
let username: string = "Prajwal";

username = "Shadow"; // ☑ allowed
// username = 100 ❌ error
```

TypeScript ensures **only strings go into name**.

3️⃣ number

In JavaScript there is **only one numeric type**: number.

It includes:

- integers
- decimals
- floats

```
let age: number = 22;
let price: number = 99.99;
let temperature: number = -10;
```

Example:

```
function add(a: number, b: number): number {
  return a + b;
}
```

4️⃣ boolean

Stores **true / false** values.

```
let isLoggedIn: boolean = true;
let isAdmin: boolean = false;
```

Example:

```
function checkLogin(status: boolean) {
  if (status) {
    console.log("User logged in");
  }
}
```

5️⃣ null

Represents **intentional absence of value**.

```
let data: null = null;
```

Used when you **explicitly want to say there is no value**.

Example:

```
let selectedUser: string | null = null;
```

Meaning:

- either string
- or null

6️⃣ undefined

Represents **variable declared but not assigned**.

```
let value: undefined = undefined;
```

Example:

```
let username: string | undefined;
```

Meaning username might **not exist yet**.

7️⃣ symbol

Used for **unique identifiers** (rare but powerful).

```
const id: symbol = Symbol("id");
```

Two symbols are always unique:

```
Symbol("id") === Symbol("id") // false
```

Used in **advanced object property control**.

8️⃣ bigint

Used for **very large numbers** beyond number limit.

```
let bigNumber: bigint = 12345678901234567890n;
```

The n at the end makes it a bigint.

Example use case:

- cryptography
- financial calculations
- scientific computing

9️⃣ Type Inference (Important Concept)

TypeScript often **infers the type automatically**.

```
let name = "Prajwal";
```

TS automatically understands:

name: string

So writing this is optional:

```
let name: string = "Prajwal";
```

Primitive types in TypeScript are the basic data types like string, number, boolean, null, undefined, symbol, and bigint. They represent simple immutable values and form the foundation for building more complex types such as objects, arrays, and interfaces.

ANY vs Unknown Vs Never

1️⃣ any – “Turn TypeScript Off”

any means **TypeScript stops checking types**.

```
let value: any = 10;

value = "hello";
value = true;
value = {};
```

All allowed ✅

You can even do dangerous things:

```
let data: any = "Prajwal";

data.toUpperCase(); // works
data(); // ❌ runtime crash but TS won't warn
```

Why it’s bad

Because you lose the **main benefit of TypeScript: type safety**.

Think of any as:

“I don't care what this value is.”

When it's used

- Migrating **JavaScript → TypeScript**
- Temporary quick fixes (not recommended long-term)

2️⃣ unknown – “Safe any”

unknown is like **a safer version of any**.

```
let value: unknown = "Prajwal";
```

You **cannot use it directly** until you check its type.

```
let value: unknown = "Prajwal";

value.toUpperCase(); // ❌ Error
```

You must **narrow the type first**.

```
if (typeof value === "string") {
  console.log(value.toUpperCase()); // ☑ safe
}
```

So TypeScript forces you to **verify the type before using it**.

Think of unknown as

“I don't know the type yet, but I will check before using it.”

Real-world example

API response:

```
function getData(): unknown {
  return JSON.parse('{"name":"Prajwal"}');
}
```

You must validate it before using.

3️⃣ never – “This should never happen”

never represents **a value that never exists**.

It appears in functions that **never return**.

Example:

```
function throwError(message: string): never {
  throw new Error(message);
}
```

This function **always throws**, so it never returns.

Another example: **infinite loop**

```
function infiniteLoop(): never {
  while (true) {}
}
```

Function Parameters and return type

1️⃣ Function Parameter Types

In TypeScript, you define the **type of each parameter**.

Syntax:

```
function functionName(param: type) {
}
```

Example:

```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Prajwal"); // ☑
greet(10); // ❌ Error
```

Here:

- name: string → parameter must be a **string**
- TS prevents wrong inputs.

2️⃣ Multiple Parameters

```
function add(a: number, b: number) {
  return a + b;
}

add(5, 10); // ☑
add(5, "10"); // ❌
```

Each parameter has its own type.

Return Type

You can explicitly define the **return type**.

Syntax:

```
function fn(): returnType {
}
```

Example:

```
function multiply(a: number, b: number): number {
  return a * b;
}
```

Here:

- a → number
- b → number
- return value → number

If you return something else:

```
function multiply(a: number, b: number): number {
return "hello"; // ❌ Type error
}
```

TypeScript Return Type Inference

TS can **automatically detect return types**.

```
function add(a: number, b: number) {
  return a + b;
}
```

TS infers:

```
add(): number
```

But in **large projects**, developers still write return types for clarity.

5️⃣ Void Return Type

Used when a function **returns nothing**.

```
function logMessage(message: string): void {
  console.log(message);
}
```

Example:

```
function print(): void {
  console.log("Hello");
}
```

6️⃣ Optional Parameters

Use ?.

```
function greet(name: string, age?: number) {
  console.log(name);
}
```

Valid calls:

```
greet("Prajwal");
greet("Prajwal", 22);
```

Rule:

⚠️ Optional parameters must be **last**.

7️⃣ Default Parameters

It sets default value when param is not passed when function called

```
function greet(name: string, country: string = "India") {
  console.log(name + " from " + country);
}

greet("Prajwal");  // Prajwal form India
greet("Prajwal", "Japan"); // Prajwal from japan
```

8️⃣ Rest Parameters

When number of arguments is unknown.

```
function sum(...numbers: number[]): number {
  return numbers.reduce((a, b) => a + b, 0);
}

sum(1,2,3,4);
```

numbers → array of numbers.

9️⃣ Arrow Function Types

Same concept applies.

```
const add = (a: number, b: number): number => {
  return a + b;
};
```

Short version:

```
const add = (a: number, b: number): number => a + b;
```

In TypeScript, we define types for function parameters and return values to ensure type safety. Parameters are typed individually, and the return type is defined after the parameter list using a colon. TypeScript can also infer return types automatically, but explicit return types improve readability and maintainability in large applications.

Types

A type alias lets you create a custom name for a type.

Syntax:

```
type TypeName = type;
```

Example:

```
type UserID = number;

let id: UserID = 101;
```

Instead of repeating complex types, you give them a **name**.

Example with object:

```
type User = {
  name: string;
  age: number;
};
```

Usage:

```
const user: User = {
  name: "Prajwal",
  age: 22
};
```

Think of type as **creating your own data structure**.

2 Union Types (|)

Union means **a value can be one of multiple types**.

Symbol:

|

Example:

```
let value: string | number;

value = "Hello";
value = 10;
```

Both are valid.

But this is **not allowed**:

```
value = true; // ❌
```

Real Backend Example

API status:

```
type Status = "success" | "error" | "loading";
```

Usage:

```
let requestStatus: Status = "success";
```

Now TS prevents invalid values:

```
requestStatus = "done"; // ❌ error
```

Union Type Narrowing

You must **check the type before using it**.

```
function printId(id: string | number) {
  if (typeof id === "string") {
    console.log(id.toUpperCase());
  } else {
    console.log(id);
  }
}
```

TS ensures safety.

3️⃣ Intersection Types (&)

Intersection combines **multiple types into one**.

Symbol:

&

```
type Person = {
name: string;
};

type Employee = {
employeeId: number;
};

type Staff = Person & Employee;
```

Now Staff must have **both properties**.

```
const staff: Staff = {
  name: "Prajwal",
  employeeId: 123
};
```

If one is missing:

```
const staff: Staff = {
  name: "Prajwal"
}; // ❌ error
```

Union vs Intersection (Key Difference)

Union (|)

Value can be **one of many types**

```
type ID = string | number;
```

Possible values:

```
"abc"
123
```

Intersection (&)

Value must satisfy **all types**

```
type Admin = User & Permissions
```

Object must contain **everything from both types**.

Union types allow a value to be one of multiple types using the | operator, while intersection types combine multiple types into a single type using the & operator. Type aliases using type help define reusable custom types, improving code readability and maintainability.

Interface

An interface defines the structure (shape) of an object.

It tells TypeScript:

“Any object of this type must have these properties.”

Syntax:

```
interface InterfaceName {
  property: type;
}
```

Example:

```
interface User {
  name: string;
  age: number;
}
```

Usage:

```
const user: User = {
  name: "Prajwal",
  age: 22
};
```

If you miss something:

```
const user: User = {
  name: "Prajwal"
};
```

❌ Error because age is missing.

2️⃣ Optional Properties

Some properties may not always exist.

Use ?.

```
interface User {
  name: string;
  age?: number;
}
```

Now both are valid:

```
const u1: User = { name: "Prajwal" };

const u2: User = {
  name: "Prajwal",
  age: 22
};
```

3️⃣ Readonly Properties

If a property should **never change**.

```
interface User {
  readonly id: number;
  name: string;
}
```

Example:

```
const user: User = {
  id: 1,
  name: "Prajwal"
};

user.id = 2; // ❌ not allowed
```

Interface for Functions

Interfaces can define **function structure**.

```
interface AddFunction {
  (a: number, b: number): number;
}
```

Usage:

```
const add: AddFunction = (x, y) => x + y;
```

Interface for Objects with Methods

```
interface User {
  name: string;
  greet(): string;
}
```

Usage:

```
const user: User = {
  name: "Prajwal",
  greet() {
    return "Hello " + this.name;
  }
};
```

6️⃣ Extending Interfaces

Interfaces support inheritance.

Example:

```
interface Person {
  name: string;
}

interface Employee extends Person {
  employeeId: number;
}
```

Usage:

```
const emp: Employee = {
  name: "Prajwal",
  employeeId: 101
};
```

Employee now has:

name
employeeId

Interface Merging (Very Unique Feature)

Interfaces with the same name merge automatically.

```
interface User {
  name: string;
}

interface User {
  age: number;
}
```

Final interface becomes:

```
interface User {
  name: string;
  age: number;
}
```

This is called declaration merging.

type cannot do this.

Real Backend Example

API response:

```
interface ApiResponse {
  success: boolean;
  message: string;
  data?: any;
}
```

Usage:

```
const response: ApiResponse = {
  success: true,
  message: "User created"
};
```

An interface in TypeScript is used to define the structure or contract of an object. It specifies the properties and methods an object must have. Interfaces support features like optional properties, readonly fields, extension (inheritance), and declaration merging, making them very useful for designing scalable applications.

Types vs Interface

Both interface and type are used to define the structure of data in TypeScript. Interfaces are mainly used for defining object shapes and support features like declaration merging and extension with extends. Types are more flexible because they can represent unions, intersections, primitives, and tuples. In practice, interfaces are preferred for object contracts and APIs, while types are used for advanced type compositions.

Arrays

What is an Array in TypeScript?

An array stores multiple values of the same type.

Example:

```
let numbers: number[] = [1, 2, 3, 4];
```

Here:

numbers → array
number[] → array of numbers

TypeScript ensures only numbers are stored.

```
numbers.push(5); // ☑
numbers.push("hello"); // ❌ Error
```

Alternative Array Syntax

There are **two ways** to type arrays.

Method 1 (Most common)

```
let users: string[] = ["Prajwal", "Alex", "John"];
```

Method 2 (Generic syntax)

```
let users: Array<string> = ["Prajwal", "Alex"];
```

Both are **exactly the same**.

Most developers prefer:

string[]

because it's shorter.

Arrays of Objects

Very common in backend APIs.

Example:

```
interface User {
  name: string;
  age: number;
}

let users: User[] = [
  { name: "Prajwal", age: 22 },
  { name: "Alex", age: 30 }
];
```

Each item must follow the **User interface**.

Mixed Arrays (Union Types)

Sometimes arrays can hold **multiple types**.

```
let values: (string | number)[] = [1, "hello", 5, "world"];
```

Meaning:

string OR number allowed

But this is invalid:

```
values.push(true); // ❌
```

Readonly Arrays

Used when array **should not change**.

```
let numbers: readonly number[] = [1, 2, 3];
```

Now this is blocked:

```
numbers.push(4); // ❌
numbers[0] = 10; // ❌
```

Alternative syntax:

```
ReadonlyArray<number>
```

Example:

let numbers: ReadonlyArray<number> = [1,2,3];

Multi-Dimensional Arrays

Arrays inside arrays.

Example:

```
let matrix: number[][] = [
  [1,2],
  [3,4],
  [5,6]
];
```

This is a **2D array**.

Tuple (Special Type of Array)

Tuple = **fixed length array with specific types**.

Example:

let user: [string, number] = ["Prajwal", 22];

Meaning:

index 0 → string
index 1 → number

Invalid:

["Prajwal", "22"] // ❌

Real Backend Example

API response:

```
interface Product {
  id: number;
  name: string;
}
```

```
const products: Product[] = [
  { id: 1, name: "Laptop" },
  { id: 2, name: "Phone" }
];
```

In TypeScript, arrays are typed collections that store multiple values of the same type. They can be defined using type[] syntax or Array<type>. TypeScript ensures type safety by preventing invalid values from being inserted. Arrays can also store objects, union types, and support advanced forms like readonly arrays and tuples.

What is an Enum?

An **enum (enumeration)** is a way to define a **group of named constants**.

Syntax:

```
enum EnumName {
  VALUE1,
  VALUE2,
  VALUE3
}
```

Example:

```
enum Status {
  Pending,
  Success,
  Failed
}
```

Usage:

```
let requestStatus: Status = Status.Success;
```

Instead of using strings like "success" everywhere, enums give you **structured constants**.

Numeric Enums (Default)

By default, enums assign **numbers starting from 0**.

```
enum Status {
  Pending,  // 0
  Success,  // 1
  Failed    // 2
}
```

Example:

```
console.log(Status.Pending); // 0
```

Custom Numeric Enums

You can assign numbers manually.

```
enum Status {
  Pending = 1,
  Success = 2,
  Failed = 3
}
```

String Enums

Instead of numbers, you can use strings.

```
enum Role {
  Admin = "ADMIN",
  User = "USER",
  Guest = "GUEST"
}
```

Usage:

```
let role: Role = Role.Admin;
```

Result:

"ADMIN"

Most backend projects prefer **string enums** because they’re clearer.

5️⃣ Example in Backend

User roles system:

```
enum UserRole {
  Admin = "ADMIN",
  Moderator = "MODERATOR",
  User = "USER"
}
```

Usage:

```
function checkAccess(role: UserRole) {
  if (role === UserRole.Admin) {
    console.log("Full access");
  }
}
```

Now TypeScript prevents invalid roles.

checkAccess("SUPERADMIN"); // ❌ error

6️⃣ Reverse Mapping (Numeric Enums Only)

Numeric enums allow reverse lookup.

```
enum Direction {
  Up,
  Down
}
```

Example:

```
console.log(Direction[0]); // "Up"
```

This **doesn't work with string enums**.

Enum vs Union Types (Important)

Many developers prefer **union types instead of enums**.

Example:

```
type Status = "pending" | "success" | "failed";
```

This is often simpler and avoids extra compiled code.

So modern TypeScript code sometimes **avoids enums**.

Enums in TypeScript are used to define a set of named constants. They improve code readability and maintainability by replacing magic values with meaningful names. TypeScript supports numeric enums, string enums, and const enums. In many modern projects, union types are sometimes used instead of enums because they are lighter and simpler.

Generics

The Problem Generics Solve

Imagine a simple function that returns the value you pass in.

```
function identity(value: any) {
  return value;
}
```

Usage:

```
identity(10)
identity("hello")
identity(true)
```

Works… but there's a problem.

Because we used any, TypeScript loses type safety.

Example:

```
let result = identity("Prajwal");

result.toUpperCase();
```

TS can't guarantee it's a string because any disables checking.

So we want:

“Return the same type that was passed in.”

That’s exactly what Generics do.

Basic Generic Function

We use a type parameter (commonly T).

```
function identity<T>(value: T): T {
  return value;
}
```

Usage:

```
identity<string>("hello")
identity<number>(10)
```

Now TypeScript knows the return type.

Example:

```
let result = identity("Prajwal");

result.toUpperCase(); // ☑ TS knows it's string
```

Here T becomes string automatically.

Type Inference (Important)

You usually don’t need to specify the type manually.

```
identity("hello")
```

TS automatically infers:

T = string

This is called generic type inference.

🔽 Generics with Arrays

Example:

```
function getFirstElement<T>(arr: T[]): T {
  return arr[0];
}
```

Usage:

```
getFirstElement([1,2,3])
// T = number

getFirstElement(["a","b","c"])
// T = string
```

This function works with any array type.

5️⃣ Multiple Generic Types

You can use multiple generics.

```
function pair<K, V>(key: K, value: V) {
  return { key, value };
}
```

Usage:

```
pair<string, number>("age", 22)
```

Result:

```
{
 key: "age",
 value: 22
}
```

But usually inference handles it.

6️⃣ Generics with Interfaces

Generics are very powerful with interfaces.

Example:

```
interface ApiResponse<T> {
  success: boolean;
  data: T;
}
```

Now you can reuse it.

User API:

```
interface User {
  name: string;
  age: number;
}
```

Response type:

```
const response: ApiResponse<User> = {
  success: true,
  data: {
name: "Prajwal",
    age: 22
  }
};
```

Another API:

```
ApiResponse<Product>
ApiResponse<Order>
```

Same structure → different data types.

Why Generics Are Powerful

Without generics you'd write multiple functions:

```
getNumber()
getString()
getUser()
```

With generics:

```
getValue<T>()
```

One function works for all types.

Benefits:

- **reusable code**
- **strong type safety**
- **flexible APIs**
- **better libraries**
-

Generics in TypeScript allow us to write reusable and type-safe components that work with multiple data types. Instead of using any, generics use type parameters like <T> to preserve type information. They are commonly used in functions, interfaces, classes, and libraries like Promise<T> and Array<T>. Generics improve flexibility while maintaining strong type safety.

1️⃣ Why use TypeScript instead of JavaScript?

Answer

TypeScript is a superset of JavaScript that adds static typing and compile-time error checking. It helps catch errors before runtime, improves code maintainability, provides better IDE support like autocomplete and refactoring, and is especially useful for large-scale applications and team collaboration.

2️⃣ How does TypeScript work internally?

Answer

TypeScript does not run directly in the browser or Node.js. It is transpiled into JavaScript using the TypeScript compiler (tsc). The compiler performs static type checking and converts TypeScript code into plain JavaScript, which is then executed by the JavaScript engine like V8.

Flow:

TypeScript → tsc compiler → JavaScript → JS Engine

3️⃣ What problems does TypeScript solve?

Answer

TypeScript solves issues like lack of type safety, runtime errors caused by incorrect data types, difficulty maintaining large codebases, and poor tooling support in JavaScript. By introducing static typing and better developer tooling, it makes code more predictable and maintainable.

4️⃣ What are primitive types in TypeScript?

Answer

Primitive types in TypeScript represent simple data values such as string, number, boolean, null, undefined, symbol, and bigint. These are the basic

building blocks used to create more complex data structures like objects, arrays, and interfaces.

5️⃣ Difference between any, unknown, and never

Answer

any

- **disables type checking**
- **allows any operation**

unknown

- **safer alternative to any**
- **must check the type before using**

never

- **represents values that never occur**
- **used in functions that never return**

Example:

```
function throwError(): never {
  throw new Error("Error");
}
```

6️⃣ What are Union Types?

Answer

Union types allow a variable to hold multiple possible types using the | operator.

Example:

```
let id: string | number;
```

This means id can be either string or number.

7️⃣ What are Intersection Types?

Answer

Intersection types combine multiple types into one using the & operator, meaning the resulting type must satisfy all combined types.

Example:

```
type User = { name: string }
type Admin = { role: string }

type AdminUser = User & Admin
```

Now the object must have both properties.

8️⃣ What is the difference between interface and type?

Answer

Both are used to define the structure of data. Interfaces are mainly used for defining object shapes and support declaration merging. Types are more flexible because they can represent unions, primitives, tuples, and intersections.

General rule:

interface → objects / APIs
type → unions / advanced types

9️⃣ What are Generics in TypeScript?

Answer

Generics allow us to create reusable and type-safe components that work with multiple data types. Instead of using any, generics use type parameters like <T> to preserve type information.

Example:

```
function identity<T>(value: T): T {
  return value;
}
```

🔟 What are Enums?

Answer

Enums are used to define a set of named constants. They improve readability and prevent the use of magic values in code.

Example:

```
enum Status {
  Pending,
  Success,
  Failed
}
```

1️⃣1️⃣ What is the difference between string[] and Array<string>?

Answer

Both represent arrays of strings. string[] is shorthand syntax, while Array<string> uses generic syntax. They behave exactly the same.

Example:

```
let users: string[]
let users2: Array<string>
```

1️⃣2️⃣ What is a Tuple?

Answer

A tuple is a fixed-length array where each position has a specific type.

Example:

```
let user: [string, number] = ["Prajwal", 22];
```

1️⃣3️⃣ What is tsconfig.json?

Answer

tsconfig.json is the configuration file for the TypeScript compiler. It defines compiler options such as target JavaScript version, module system, source directories, and strict type checking rules.

Example options:

target
module
rootDir
outDir
strict

1️⃣4️⃣ How do you run a TypeScript project?

Answer

Steps:

1. npm init
2. install typescript
3. create tsconfig.json
4. write .ts files
5. compile using tsc
6. run compiled JS with node

For development:

nodemon + ts-node

1️⃣5️⃣ What are Generic Constraints?

Answer

Generic constraints restrict the types that a generic parameter can accept using the extends keyword.

Example:

```
function getLength<T extends { length: number }>(value: T) {
  return value.length;
}
```

Now only values with a length property are allowed.